

COMP 356
Programming Language Structures
Notes for Chapter 7 of *Concepts of Programming Languages*
Expressions and Assignment Statements

1 Expressions

1.1 Conditional Expressions

Conditional expressions (also called **if expressions**) in C, C++ and Java are similar to if-then-else expressions in denotational semantics. The expression:

`<expr0> ? <expr1> : <expr2>`

evaluates to `<expr1>` if `<expr0>` is true, and `<expr2>` if `<expr0>` is false. For example:

`a = (b < 0) ? -b : b;`

places the absolute value of `b` into `a`.

Formally, the semantics of the C conditional expression (ignoring side effects) is:

$$\begin{aligned} \text{Mexpr}(\langle \text{expr}_0 \rangle ? \langle \text{expr}_1 \rangle : \langle \text{expr}_2 \rangle, s) = \\ \text{if } s = \perp \text{ then } \perp \\ \text{else if } \text{Mexpr}(\langle \text{expr}_0 \rangle, s) = \perp \text{ then } \perp \\ \text{else if } \text{Mexpr}(\langle \text{expr}_0 \rangle, s) = 0 \text{ then } \text{Mexpr}(\langle \text{expr}_2 \rangle, s) \\ \text{else } \text{Mexpr}(\langle \text{expr}_1 \rangle, s) \end{aligned}$$

A conditional expression can be used anywhere an expression can be used.

1.2 Operand Evaluation Order

A subprogram (procedure or function) has **side effects** if it modifies one of its parameters or a global variable. In the presence of side effects, the order that operands of an expression are evaluated in can effect the value of the expression. For example:

```
#include <iostream>

int x = 3;

int foo(void) {
    x = 7;
    return 5;
}

int main(void) {

    std::cout << x + foo() << std::endl;
}
```

In mathematics, the order that the operands of a `+` are evaluated in does not matter. However, in this program, when the addition in `main` is evaluated, there are two possible results:

- if the call to `foo()` is evaluated before `x`, the value printed is 12.
- if `x` is evaluated before the call to `foo()`, the value printed is 8.

In fact, under g++ this program prints 12.

This violates **referential transparency** - a mathematical principle which states that calling the same function with the same parameters should always give the same result.

One way of resolving the ambiguity is to fix an order of evaluation for expression operands. For example, in Java operands are always evaluated from left to right (which would cause the program above to print 8). However, fixing an evaluation order disallows a number of compiler optimizations, so C and C++ do not fix an order.

1.3 Operator Overloading

An operator is **overloaded** if it is bound to multiple definitions. When the operator is used in an expression, which definition to use is determined based on the number and type of the operands. Examples:

- in Java, + is used both for addition and string concatenation
- in almost all programming languages, the arithmetic operators (+, -, *, ...) are overloaded to work on both integer and floating point types. Note that integer addition and floating point addition are different operations in assembly language.

Notes on overloading:

- many languages (C++, Ada, C#) allow the programmer to overload operators
- Java does not
- overloading can help program readability if the overloaded meaning of the operator is similar to the standard meaning
- overloading can hurt program readability if the overloaded meanings of the operator are very different from each other and/or the standard meaning
- overloading can reduce reliability by weakening the compiler's ability to detect type errors
- by definition, overloading must always be resolved statically. The compiler must be able to determine which meaning is meant by the (static) types of the operands.

1.4 Type Conversions

Definitions:

- a type conversion is **narrowing** if a value is converted to a type that can not hold all values of its original type. For example, converting `float` to `int` is a narrowing conversion.
- a type conversion is **widening** if a value is converted to a type that can hold at least all values of its original type. For example, converting `int` to `float` is a widening conversion.
- a **coercion** is an implicit type conversion (done automatically by the compiler)
- a **cast** is an explicit type conversion done by the programmer

Coercions are typically used to implement **mixed-mode** expressions and assignment statements. For example, adding an `int` and a `float` is allowed in C++, and the type system of the language specifies that the `int` is coerced to `float` so that the expression can be executed. Similarly, an `int` can be assigned to a variable of type `float`.

ANSI C++ includes a number of new casting functions (`static_cast()`, `dynamic_cast()`, ...) that address some of the readability and safety problems of traditional C++ casts.

1.5 Short-Circuit Evaluation

In C, C++ and Java the boolean operators `&&` and `||` are **short-circuit** operators, which means that their second operands are not evaluated if their value is not needed. This is often convenient for preventing invalid expressions from being executed. For example, in C++ the code for searching a linked list for a particular value can be written as:

```
while (ptr != NULL && ptr->info != value) ptr = ptr->next;
```

with no concern that a null pointer will be dereferenced when checking the loop condition. In a language without short-circuit boolean operators, this must be written as:

```
bool done = false;
while (ptr != NULL && !done) {
    if (ptr->info == value) done = true;
    else ptr = ptr->next;
}
```

Ada includes traditional and short-circuit boolean operators.

The only advantage of traditional boolean operators is that they ensure that all operands are always evaluated. This only matters when the operands have side effects, for example:

```
if (a < b || ++b > 7) a = b;
```

This statement may or may not increment `b`, so following code that relies on an increment of `b` is incorrect.

2 Assignment Statements

C, C++ and Java have a rich set of assignment operators. For example, the following all increment `count`:

```
count = count + 1;
count += 1;
count++;
++count;
```

These languages also include assignment statements with conditional targets. For example:

```
flag ? x : y = 0;
```

which is equivalent to:

```
if (flag) x = 0;
else y = 0;
```

In these same languages, assignment statements can be used as expressions. The value of the expression is the value assigned to the RHS, so (for example):

```
double d;
std::cout << (d = 3)/4 << std::endl;
```

prints 0.75 instead of 0.

A more typical use is reading until EOF (in C) as follows:

```
while ((ch = getchar()) != EOF) { ... }
```

The increment and decrement operators in C, C++ and Java explicitly combine an assignment with a particular form of expression. For example:

```
int x, y;
y = 0;
x = ++y;
// here, x and y are both 1 (pre-increment)
```

```
int x, y;
y = 0;
x = y++;
// here, x is 0 and y is 1 (post-increment)
```

In a “standard” use of these operators such as:

```
count++;
```

the result as an expression is ignored. Implications:

- `3;`
is a valid statement (which does nothing).
- the return value of a method can be ignored, i.e.:

```
String inp = JOptionPane.showInputDialog("Enter a number");
Double.parseDouble(inp);
```

is legal, even though the return value from `Double.parseDouble()` is ignored

- overloading can NOT be resolved based on the return type of methods

Using any form of assignment in an expression context makes code more difficult to read and is a frequent source of errors. Additionally, allowing assignment statements as expressions makes C and C++ much less reliable, because an error such as:

```
if (x = y) { // intended: x == y
```

gives unpredictable executions and is difficult to find. In Java and C#, only expressions of type `boolean` can be used in conditions, so this error is detected by the compiler unless `x` is of type `boolean`.

C and C++ allow mixed-mode assignments and perform arbitrary coercions of numeric types (including narrowing conversions) to implement such assignments. For example:

```
double d = 3.24259;
int i;

i = d;
```

is legal, although the compiler may generate a warning. In Java and C#, mixed-mode assignments are allowed, but only if the implied coercion is a widening conversion. Hence, the above code is not legal in Java, but the following is:

```
int i = 3;
double d;

d = i;
```

Allowing only widening conversions greatly increases reliability while retaining some of the flexibility of mixed-mode assignments.